Projects," breaking the test. You're often better served by testing something that's slightly insulated from surface changes, such as a DOM ID:

```
it "shows the project section" do
  get :dashboard
  expect(response).to have_selector("h2#projects")
end
```

The basic issue here is not limited to view testing. There are areas of model testing in which testing to a surface feature might be brittle in the face of trivial changes to the model (as opposed to tests that are brittle in the face of changes to the test data itself, which I've already discussed).

The other side of truthfulness is not just a test that fails when the logic is good, but a test that stubbornly continues to pass even if the underlying code is bad—a tautology, in other words.

Speaking of tautologies, mock objects have their own special truthfulness issues. It's easy to create a tautology by using a mock object. It's also easy to create a brittle test because a mock object often creates a hard expectation of what methods will be called on it. If you add an unexpected method call to the code being tested, you can get mock-object failures simply because of that unexpected call. I've had changes to a login filter cause hundreds of test failures because mock users going through the login filter bounced off the new call.

What You've Done

In this chapter you learned about writing effective and valuable test suites. Writing effective test suites is hard. The complexity and runtime of a typical test suite increases slowly, and often you don't realize you have a serious problem until it's too late. Paying attention to detail in keeping individual tests simple and fast will pay off over time.

The goal of your test suite is to allow you to use tests to improve the design and for the existing tests to empower you to make changes with less fear of introducing new bugs. If your test suite is slow, complicated, or fragile, then you make your tests unable to help you with either of these goals. In the worst case, you can find yourself still needing to maintain a test suite but having that test suite slow down development because of how hard it is to keep the tests synchronized with the code.

Being strict about writing tests first, writing tests against behavior and not implementation, and taking time to make the tests simple and fast will all help keep your test suite useful and healthy over the course of a long project.